

On the hardness of recognizing graphs of small mim-width and its variants

Max Dupré la Tour

Manuel Lafond

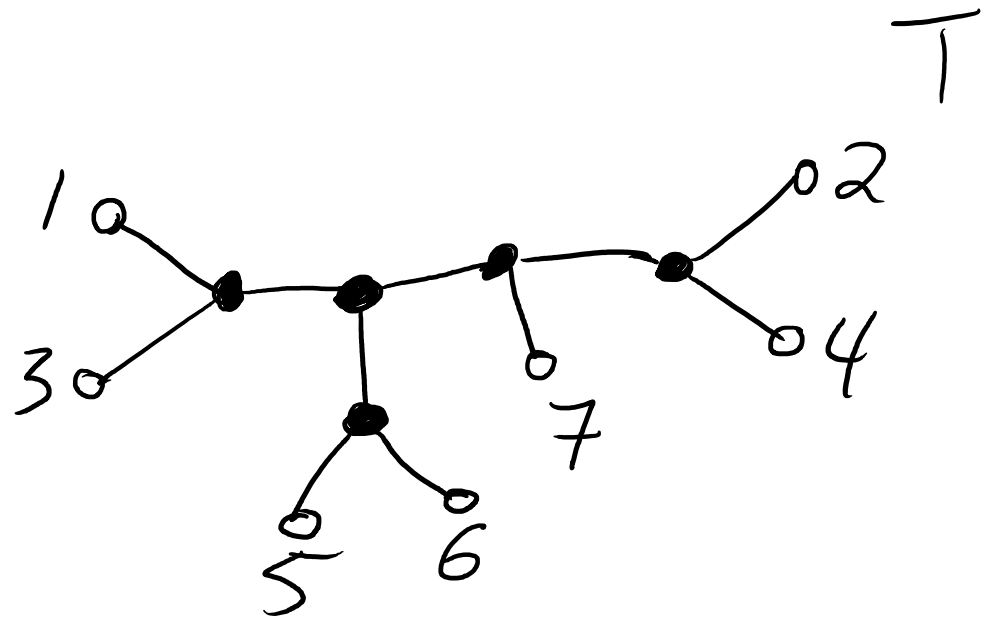
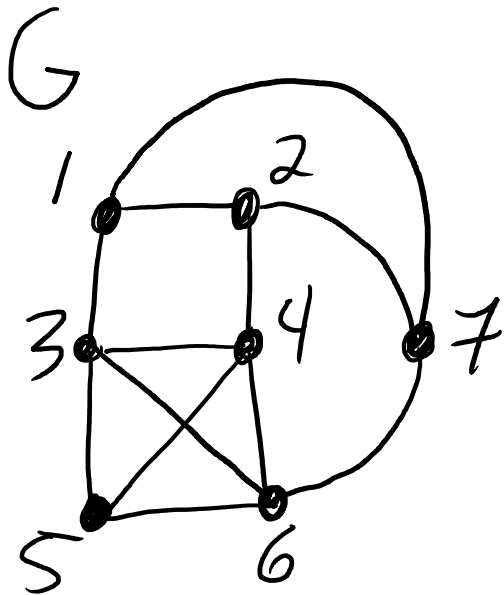
Ndiamé Ndiaye

McGill University

Université de Sherbrooke

McGill University

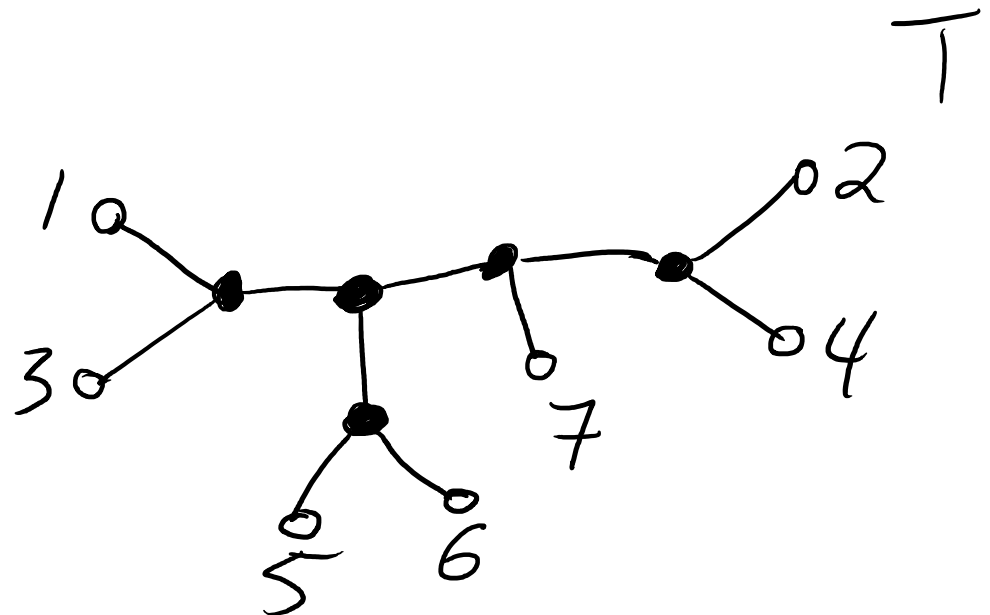
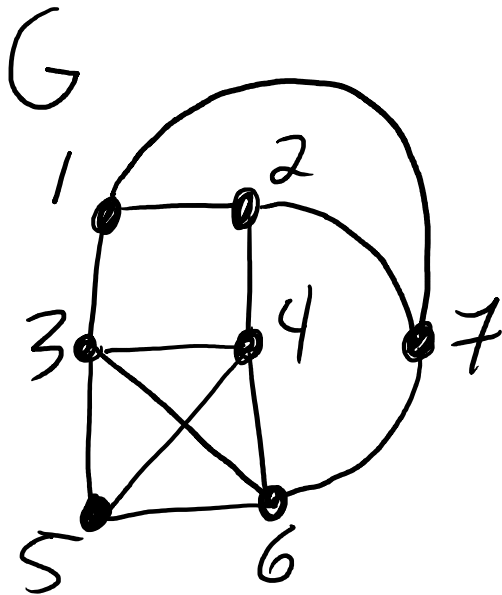
A branch decomposition of a graph G is a ternary tree T with $\text{leaves}(T) = V(G)$.



A branch decomposition of a graph G is a ternary tree T with $\text{leaves}(T) = V(G)$.

Each cut-edge e of T splits the leaves into two parts A_e, B_e .

Bipartite graph $G[A_e: B_e]$ has edges $\{uv \in E(G): u \in A_e, v \in B_e\}$

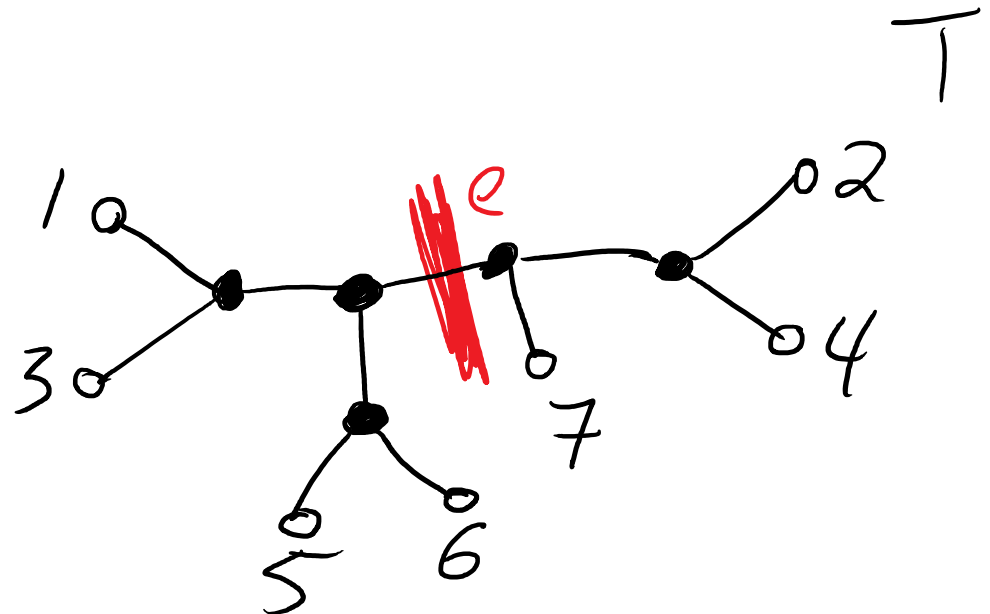
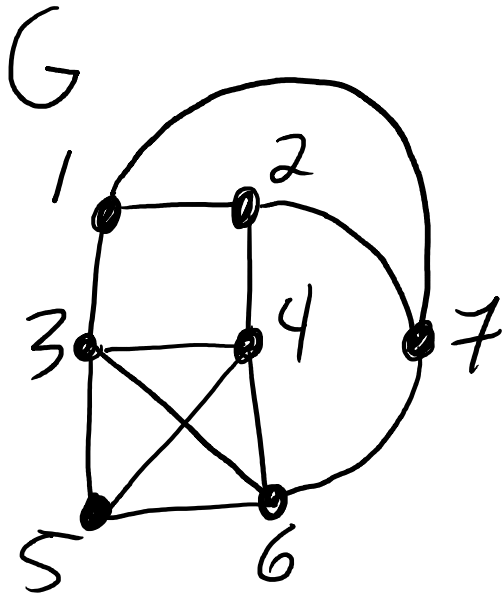
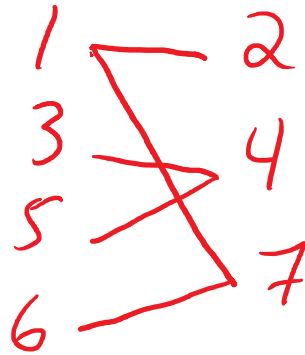


A branch decomposition of a graph G is a ternary tree T with $\text{leaves}(T) = V(G)$.

Each cut-edge e of T splits the leaves into two parts A_e, B_e .

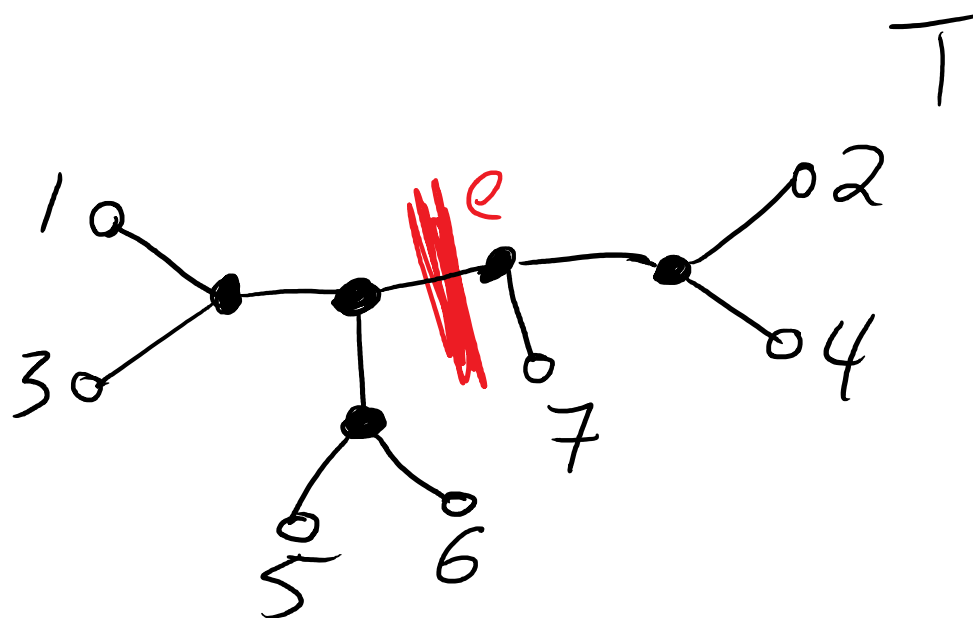
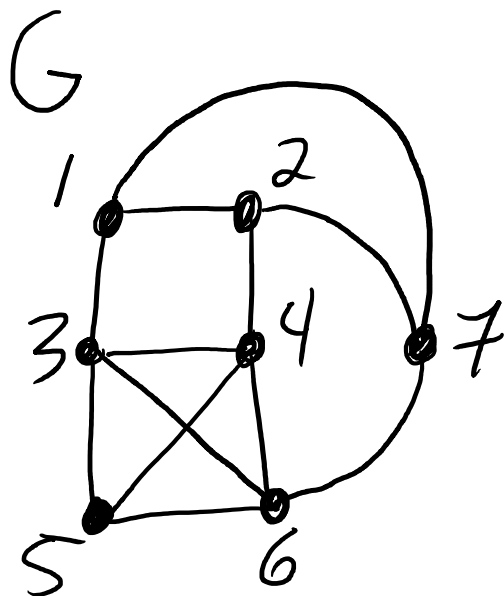
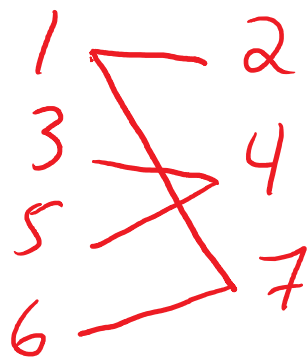
Bipartite graph $G[A_e: B_e]$ has edges $\{uv \in E(G): u \in A_e, v \in B_e\}$

$G[A_e; B_e]$



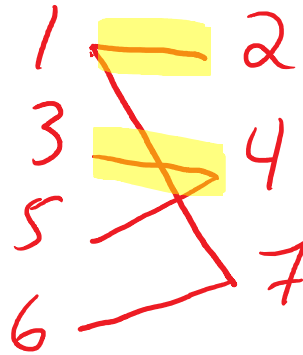
- $mim(H) = \text{size of a maximum induced matching of a graph } H$.

$G[A_e; B_e]$

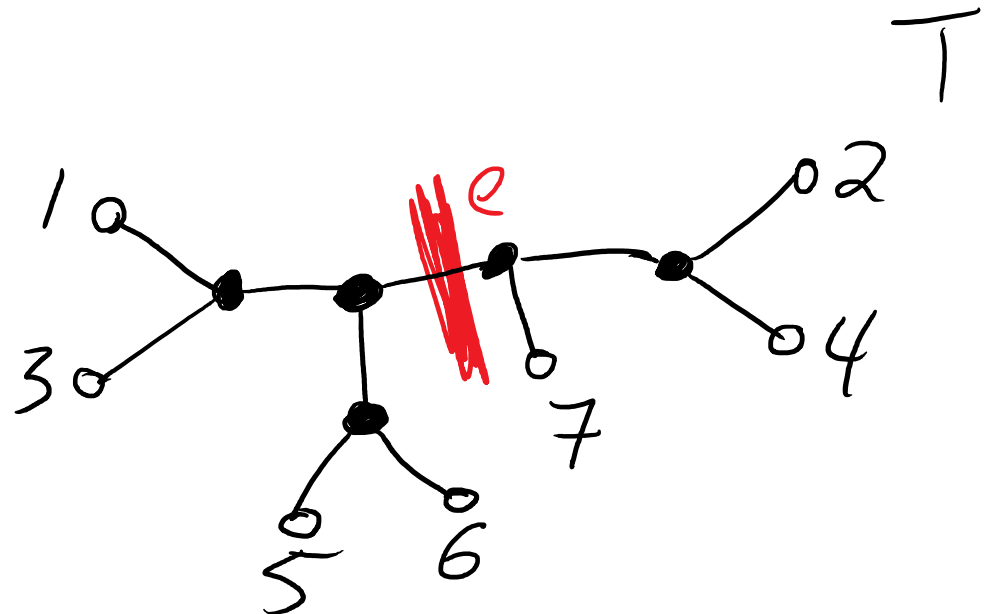
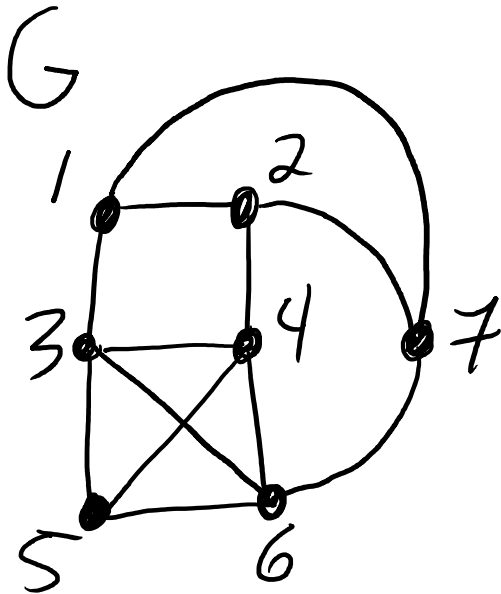


- $mim(H) = \text{size of a maximum induced matching of a graph } H$.

$G[A_e; B_e]$

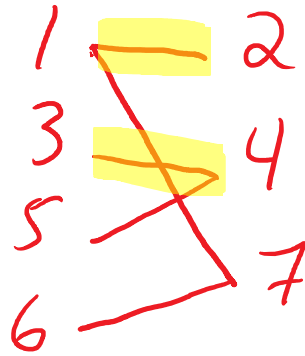


$mim(G[A_e; B_e]) = 2$

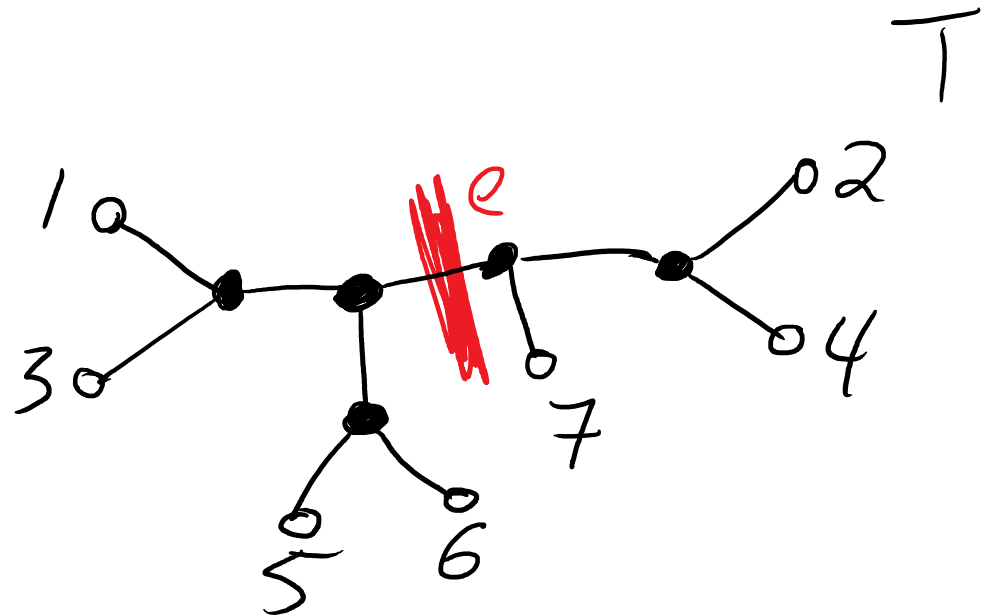
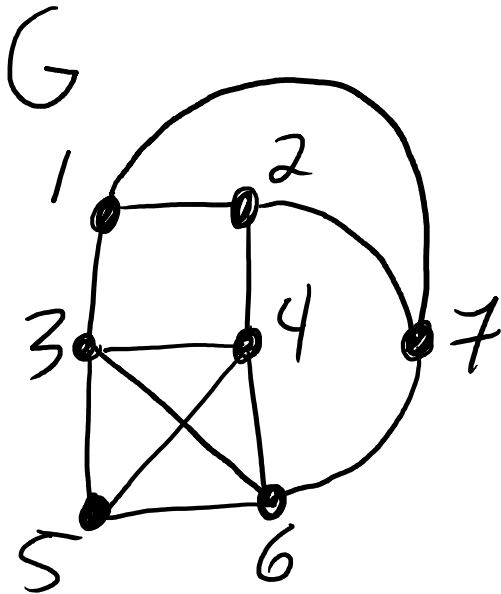


- $mim(H)$ = size of a maximum induced matching of a graph H .
- $mimwidth(T) = \max_{e \in E(T)} mim(G[A_e : B_e])$

$G[A_e : B_e]$

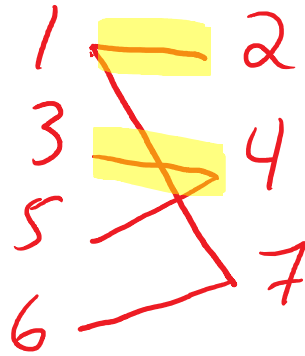


$mim(G[A_e : B_e]) = 2$

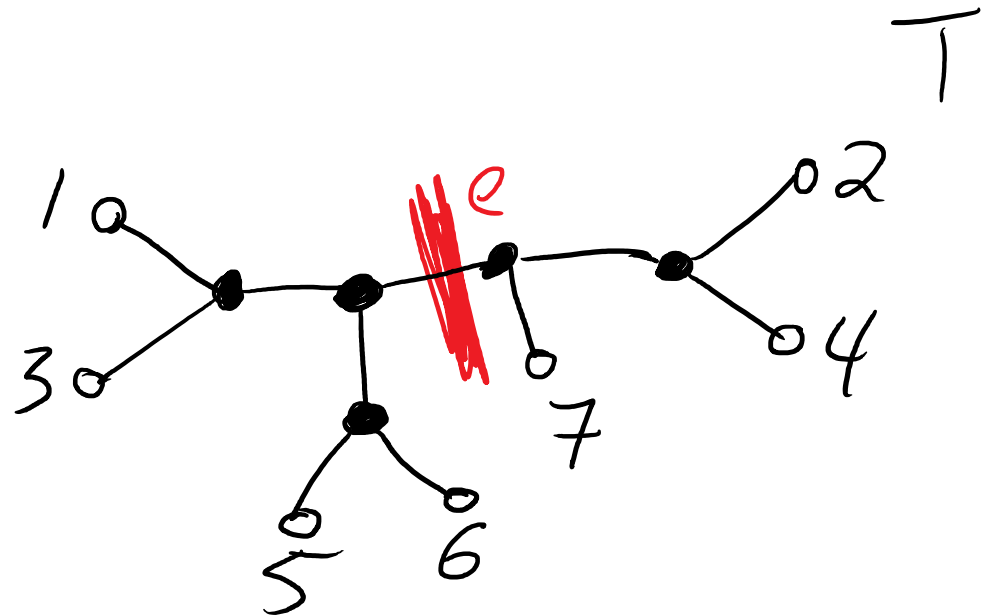
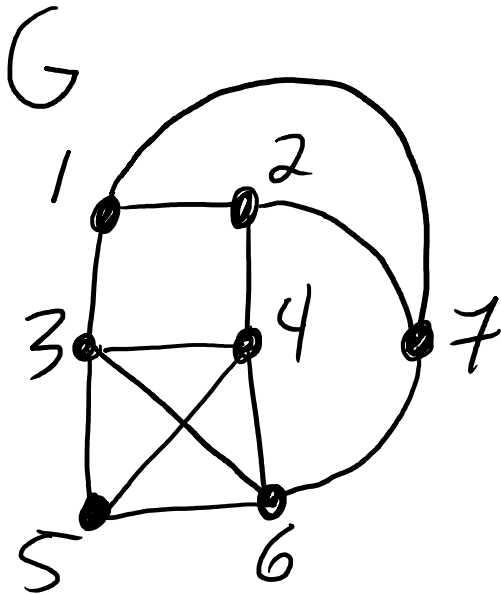


- $mim(H)$ = size of a maximum induced matching of a graph H .
- $mimwidth(T) = \max_{e \in E(T)} mim(G[A_e : B_e])$
- $mimwidth(G) = \min_T mimwidth(T)$

$G[A_e : B_e]$



$mim(G[A_e : B_e]) = 2$



- *mimwidth* introduced in [Vatshelle, 2012]
- Several problems are polytime on graphs on bounded *mimwidth*
 - e.g., longest induced path, feedback vertex set [Jaffke et al., 2019-2020]
 - meta-theorem for *mim-width* [Bergougnoux, SODA2023]
- Given G, w , NP-hard to decide if $mimwidth(G) \leq w$.
- In fact, W[1]-hard in parameter w [Sæther, Vatshelle, TCS2016]

- *mimwidth* introduced in [Vatshelle, 2012]
- Several problems are polytime on graphs on bounded *mimwidth*
 - e.g., longest induced path, feedback vertex set [Jaffke et al., 2019-2020]
 - meta-theorem for *mim-width* [Bergougnoux, SODA2023]
- Given G, w , NP-hard to decide if $mimwidth(G) \leq w$.
- In fact, W[1]-hard in parameter w [Sæther, Vatshelle, TCS2016]
- Left open: **is *mimwidth* in XP parameterized by w ?**
 - i.e., is there an $n^{f(w)}$ time algorithm to decide whether $mimwidth(G) \leq w$?
 - Question asked numerous times, [Bergougnoux et al, 2022-2023], [Otachi et al., WADS2024], ...

Question finally answered (not by us):

Theorem [Bergougnoux, Bonnet, Duronin, ICALP2025]

It is NP-hard to decide if a given graph G satisfies $mimwidth(G) \leq 1211$.

Computing *mimwidth* is therefore para-NP-hard in *mimwidth*.

Question finally answered (not by us):

Theorem [Bergougnoux, Bonnet, Duronin, ICALP2025]

It is NP-hard to decide if a given graph G satisfies $mimwidth(G) \leq 1211$.

Computing $mimwidth$ is therefore para-NP-hard in $mimwidth$.

More accurate (and stronger) statement:

Theorem [Bergougnoux, Bonnet, Duronin, ICALP2025]

It is NP-hard to distinguish whether a given graph G satisfies $linear-mimwidth(G) \leq 1211$ or $simwidth(G) \geq 1216$.

Question finally answered (not by us):

Theorem [Bergougnoux, Bonnet, Duronin, ICALP2025]

It is NP-hard to decide if a given graph G satisfies $mimwidth(G) \leq 1211$.

Computing $mimwidth$ is therefore para-NP-hard in $mimwidth$.

More accurate (and stronger) statement:

Theorem [Bergougnoux, Bonnet, Duronin, ICALP2025]

It is NP-hard to distinguish whether a given graph G satisfies $linear-mimwidth(G) \leq 1211$ or $simwidth(G) \geq 1216$.

Implies para-NP-hardness of all parameters sandwiched between $simwidth$ and $linear-mimwidth$:

- (linear) sim -width
- (linear) one-sided mim -width
- (linear) mim -width

Linear mim-width

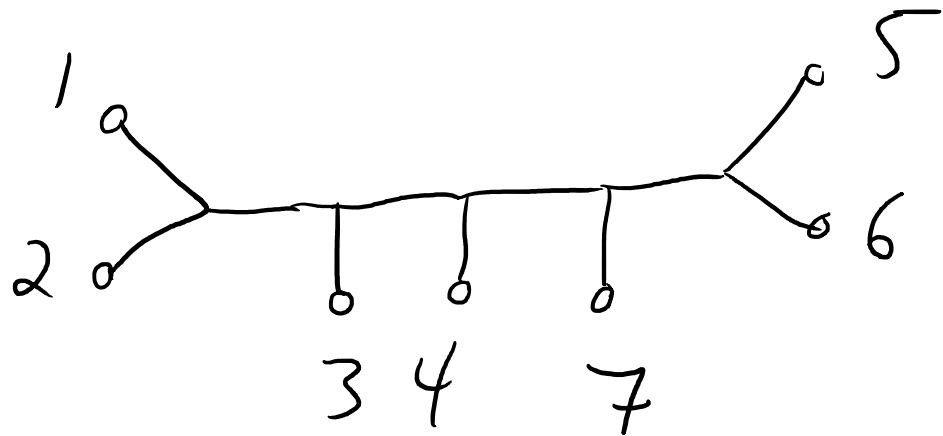
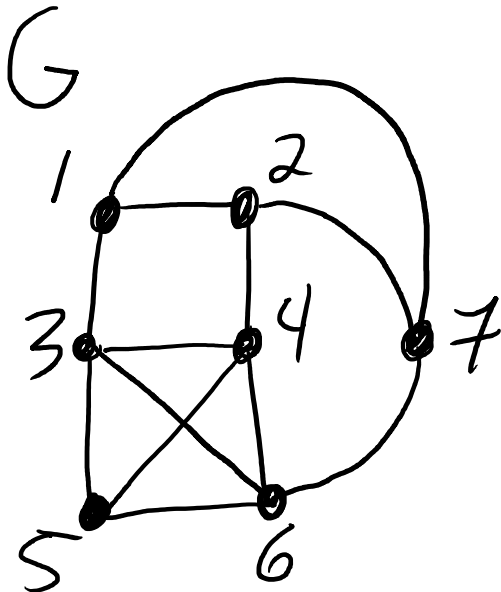
Denote by $C(G)$ the set of ternary caterpillars with leaf set $V(G)$.

- Caterpillar: tree obtained from a path by attaching leaves.

$mim(H)$ = size of a maximum induced matching of a graph H .

$$mimwidth(T) = \max_{e \in E(T)} mim(G[A_e : B_e])$$

$$linear-mimwidth(G) = \min_{T \in C(G)} mimwidth(T)$$

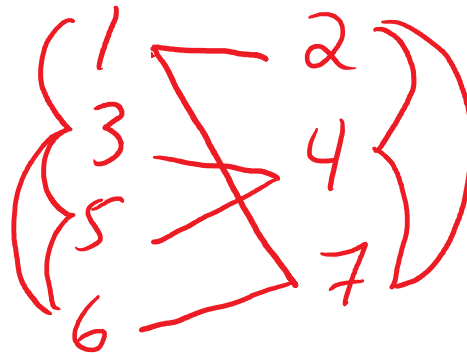
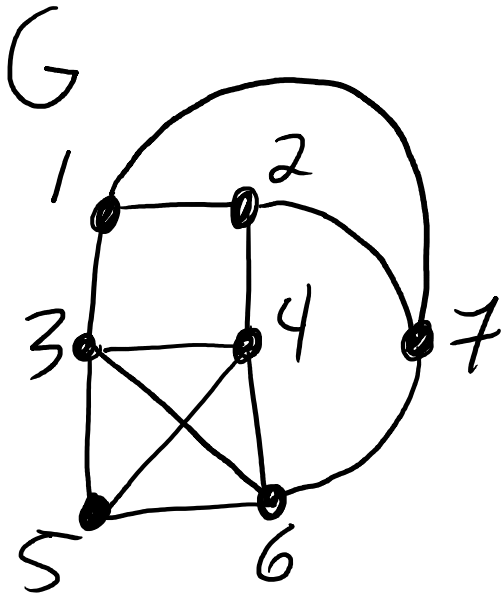


Simwidth: like mimwidth, but keep edges inside of A_e and B_e when considering the induced matching (the matching must still have ends in A_e and B_e)

$\text{sim}(A_e, B_e)$ = size of max induced matching of $G[A_e : B_e]$ that is also an induced matching of G .

$$\text{simwidth}(T) = \max_{e \in E(T)} \text{sim}(A_e, B_e)$$

$$\text{simwidth}(G) = \min_T \text{simwidth}(T)$$



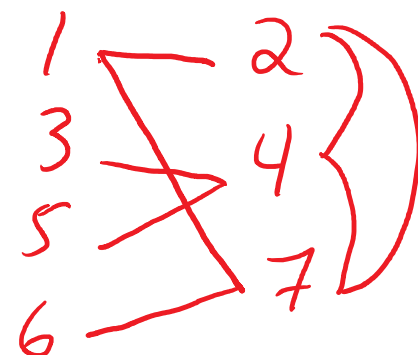
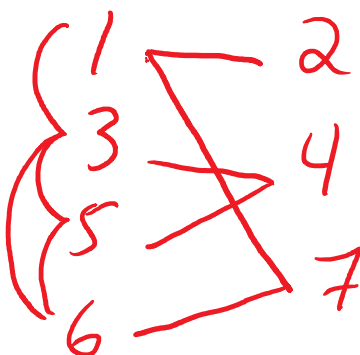
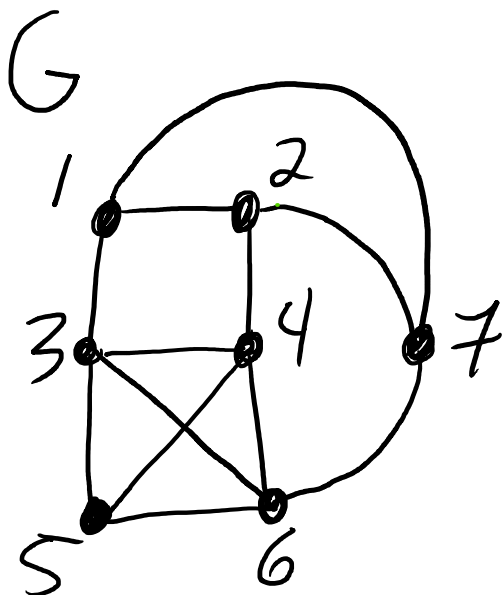
omimwidth: stands for "one-sided mim-width".

Keep edges inside of A_e or B_e , not both. Take the min.

$omim(A_e, B_e)$ = size of max induced matching of $G[A_e : B_e]$ that is also an induced matching of $G - E(G[A_e])$.

$omimwidth(T) = \max_{e \in E(T)} \min(omim(A_e, B_e), omim(B_e, A_e))$

$omimwidth(G) = \min_T omimwidth(T)$



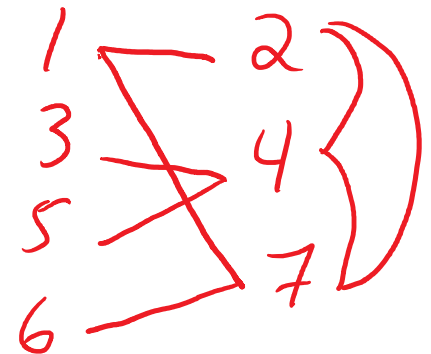
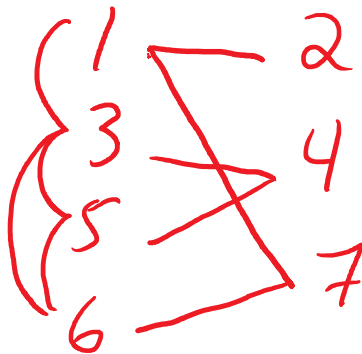
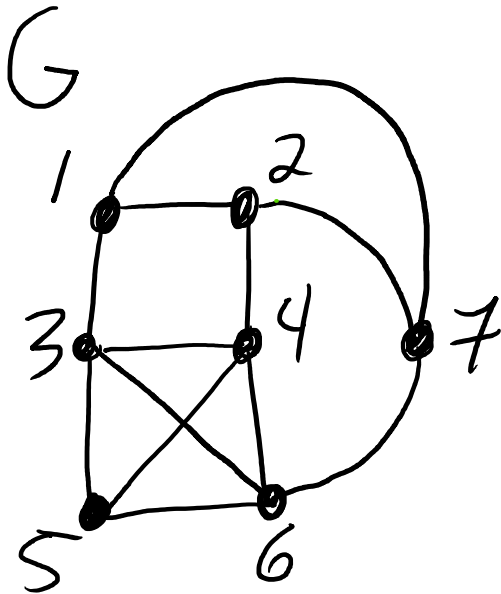
Omimwidth: stands for "large one-sided mim-width".

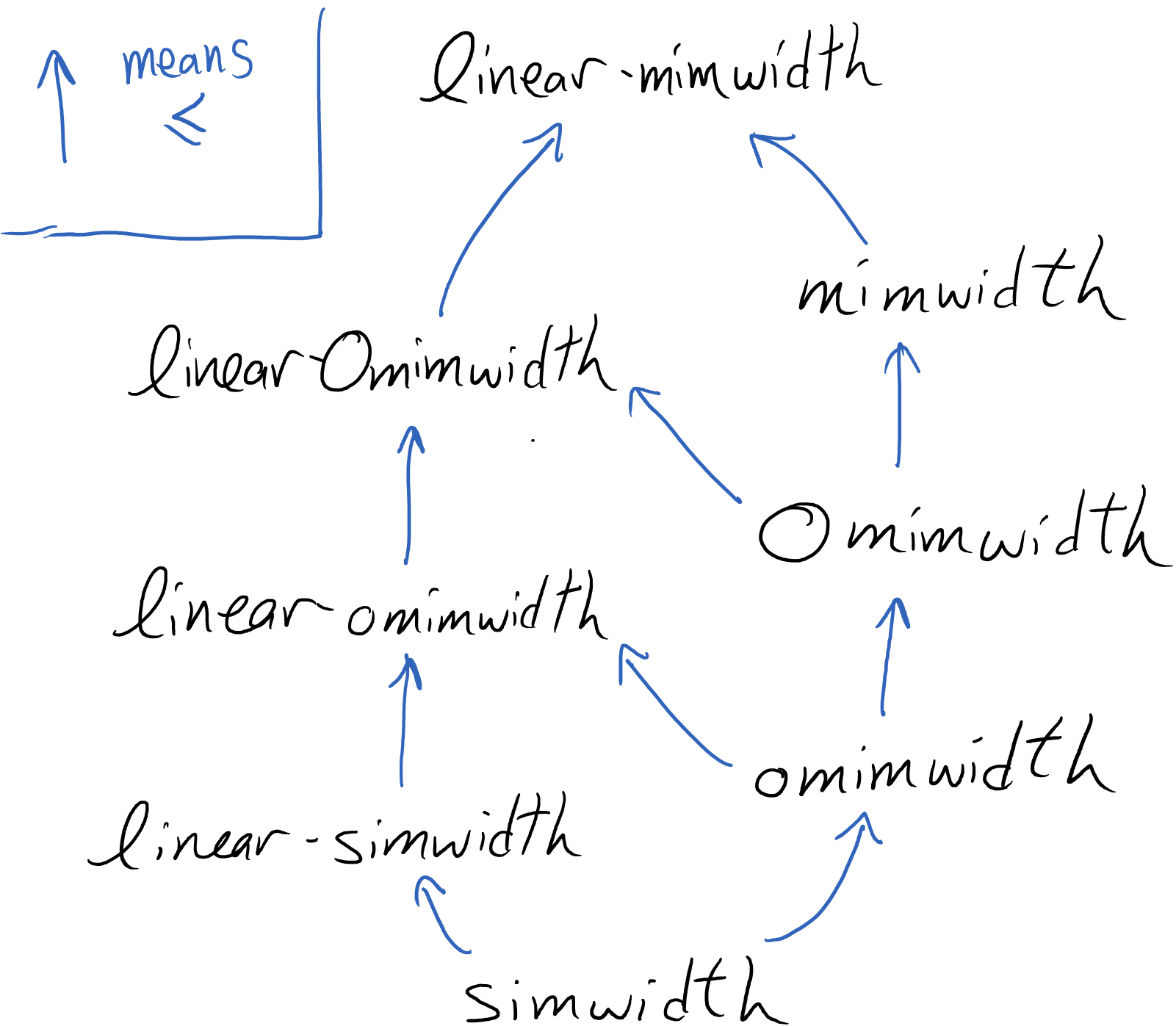
Keep edges inside of A_e or B_e , not both. Take the max.

$omim(A_e, B_e)$ = size of max induced matching of $G[A_e : B_e]$ that is also an induced matching of $G - E(G[A_e])$.

$Omimwidth(T) = \min_{e \in E(T)} \max(omim(A_e, B_e), omim(B_e, A_e))$

$Omimwidth(G) = \min_T Omimwidth(T)$





Left open: complexity of these parameters when small?

- *mimwidth* matters most when it is small, since algorithms parameterized by *mimwidth* have complexity $n^{O(w)}$.
- Would be nice to have complexity for *mimwidth* 1 or 2.

Left open: complexity of these parameters when small?

- *mimwidth* matters most when it is small, since algorithms parameterized by *mimwidth* have complexity $n^{O(w)}$.
- Would be nice to have complexity for *mimwidth* 1 or 2.

Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

For each $\sigma \in \{\textit{simwidth}, \textit{omimwidth}, \textit{Omimwidth}\}$, it is NP-hard to decide whether a given graph G satisfies $\sigma(G) = 1$.

It is also NP-hard to decide whether $\textit{linear-}\sigma(G) = 1$.

Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

It is NP-hard to decide whether a given graph G satisfies $\textit{mimwidth}(G) \leq 2$.

It is also NP-hard to decide whether $\textit{linear-mimwidth}(G) \leq 2$.

↑ means
 $\leq$

Linear-mimwidth

≤ 2 is
hard

mimwidth

linear-omimwidth

omimwidth

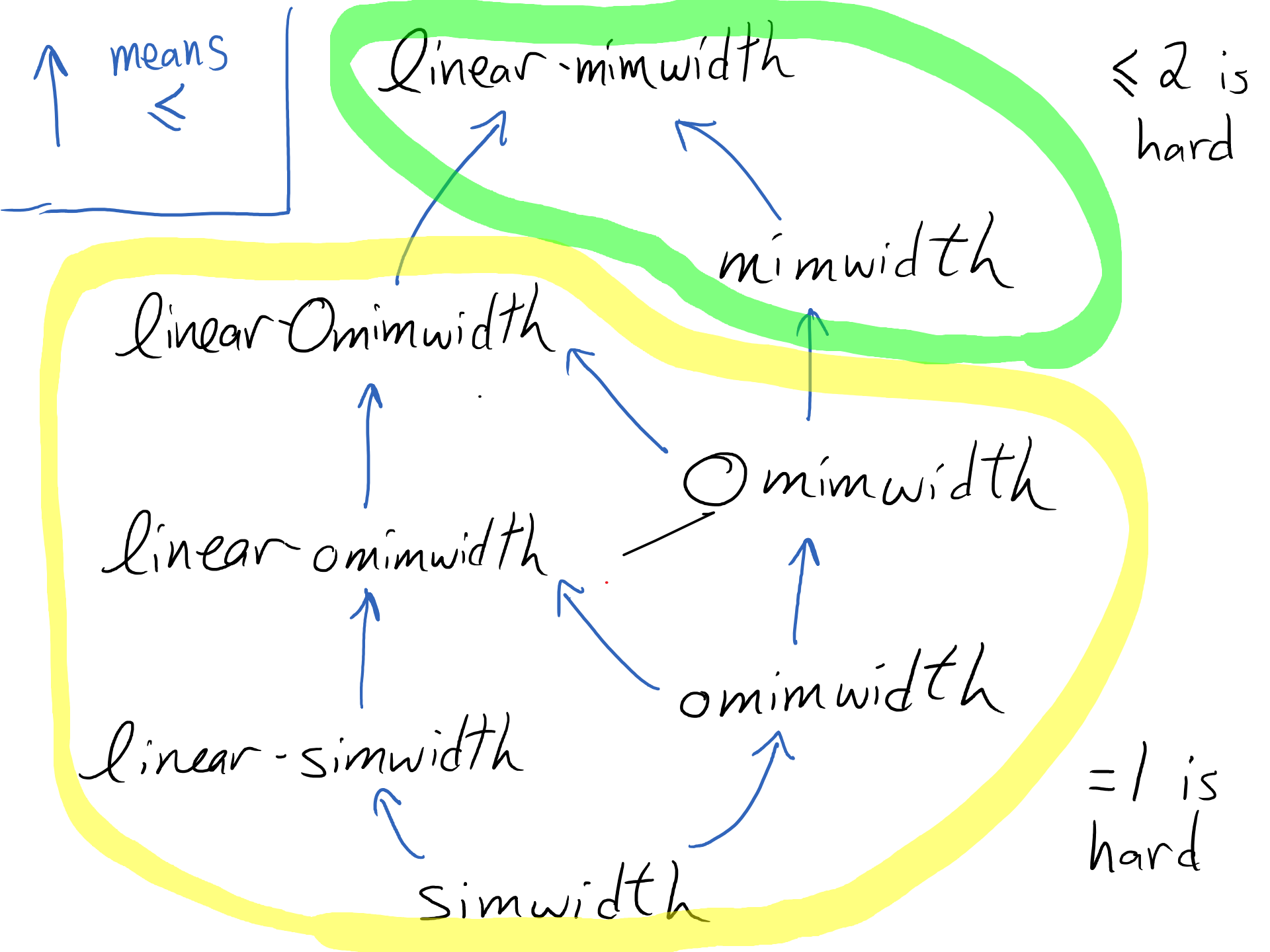
linear-omimwidth

omimwidth

linear-simwidth

simwidth

$= 1$ is
hard



Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

For each $\sigma \in \{\textit{simwidth}, \textit{omimwidth}, \textit{Omimwidth}\}$, it is NP-hard to decide whether a given graph G satisfies $\sigma(G) = 1$.

It is also NP-hard to decide whether $\textit{linear-}\sigma(G) = 1$.

Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

For each $\sigma \in \{\text{simwidth}, \text{omimwidth}, \text{Omimwidth}\}$, it is NP-hard to decide whether a given graph G satisfies $\sigma(G) = 1$.

It is also NP-hard to decide whether $\text{linear-}\sigma(G) = 1$.

Reduction from the Unrooted Quartet Consistency (UQC) problem.

Given a set X , a quartet on X is a pair of pairs $\{\{a, b\}, \{c, d\}\}$, with a, b, c, d distinct.

Written as $ab \mid cd$

A tree T satisfies $ab \mid cd$ if the $a - b$ path does not intersect the $c - d$ path.

UQC problem (Unrooted Quartet Consistency)

Input: a set Q of quartets on X

Find: a tree T with leaves X that satisfies every quartet of Q ?

Deciding if T exists is NP-complete [Steel, 1993].

UQC

ab/cd

ad/cf

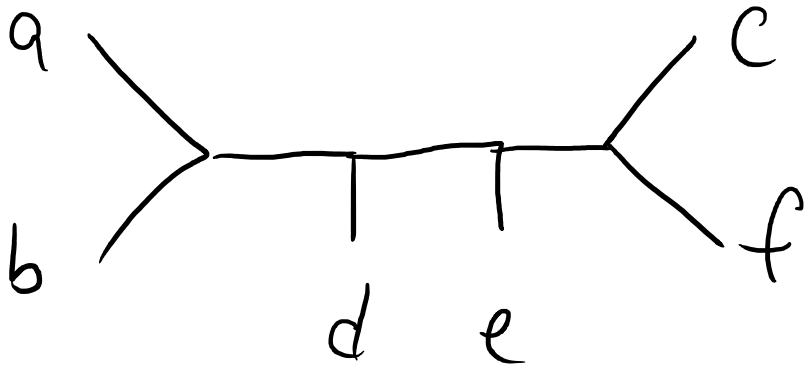
bd/ef

de/cf

UQC

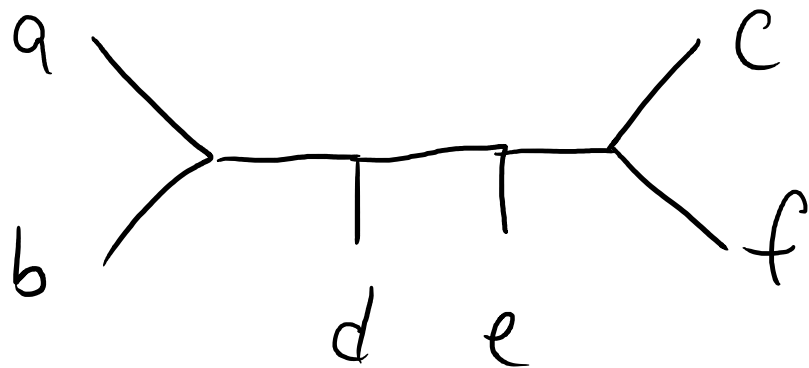
$ab|cd$ $ad|cf$

$bd|ef$ $de|cf$

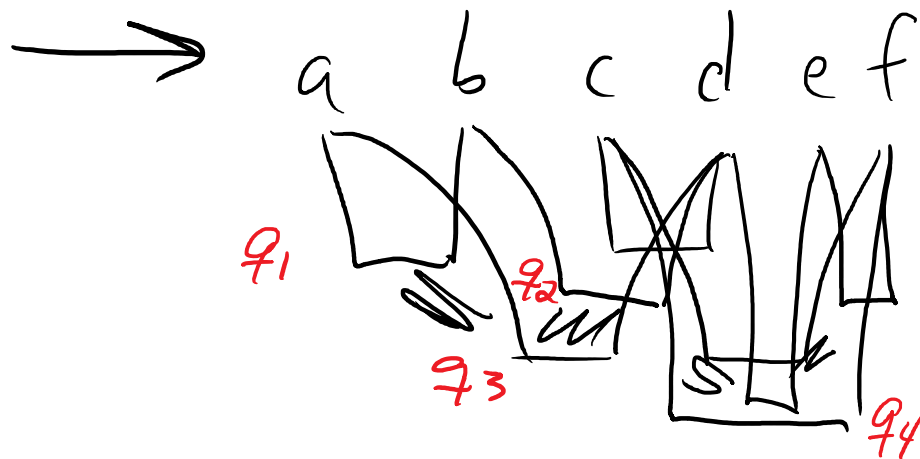


UQC

q_1 $ab|cd$ q_3 $ad|cf$
 q_2 $bd|ef$ q_4 $de|cf$

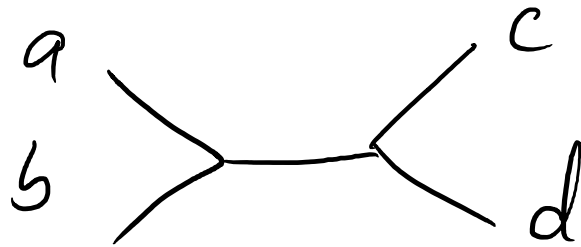
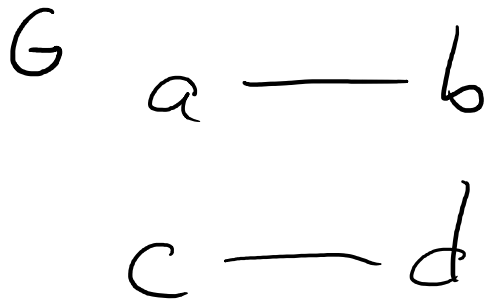


G

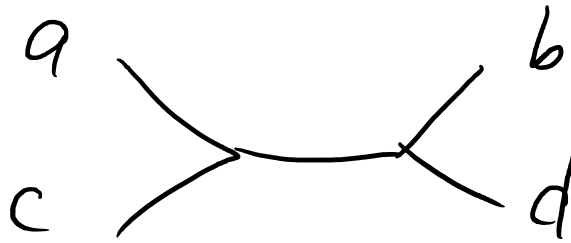


- UQC yes
 $\Rightarrow \text{linear-omimw} = 1$
- UQC no
 $\Rightarrow \text{simwidth} \geq 2$

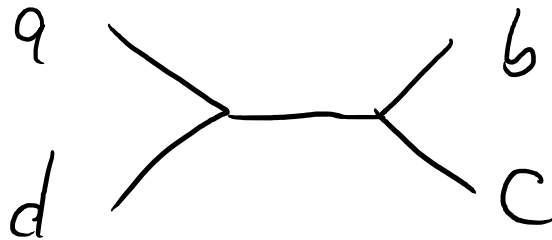
Observation: if G has *simwidth* 1 and ab, cd are independent edges of G , then any tree T for G satisfies $ab \mid cd$.



$\begin{pmatrix} a & c \\ b & d \end{pmatrix} \checkmark$



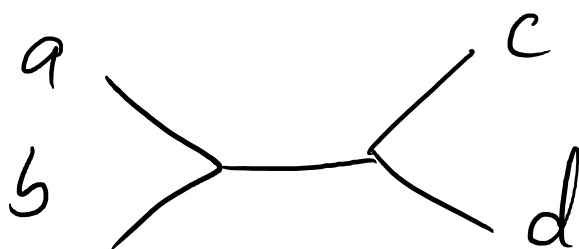
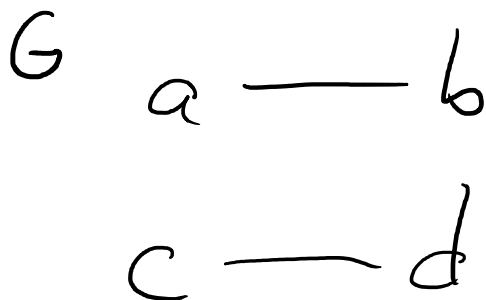
$\begin{matrix} a-b \\ c-d \end{matrix} \times$



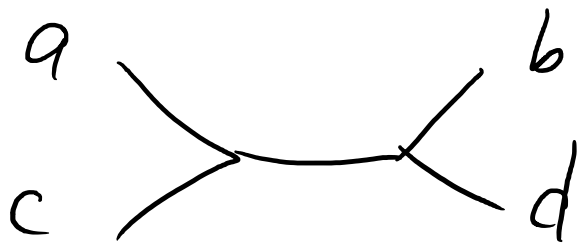
$\begin{matrix} a & b \\ d & c \end{matrix} \times \times$

Observation: if G has *simwidth* 1 and ab, cd are independent edges of G , then any tree T for G satisfies $ab \mid cd$.

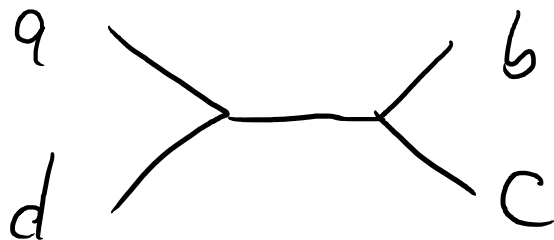
Wish: each quartet = a $2K_2$



$\begin{pmatrix} a & c \\ b & d \end{pmatrix}$ ✓



$a-b$
 $c-d$ ✗



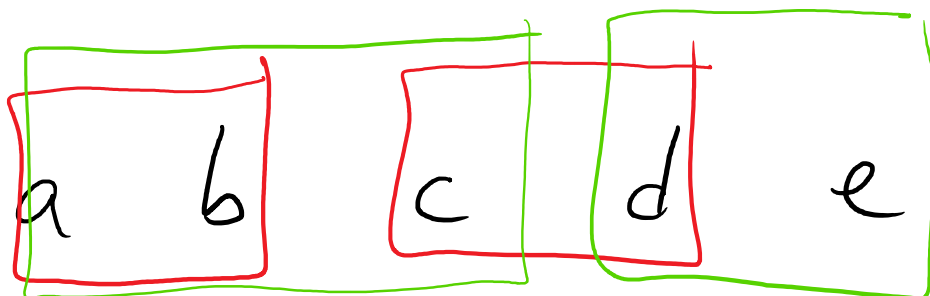
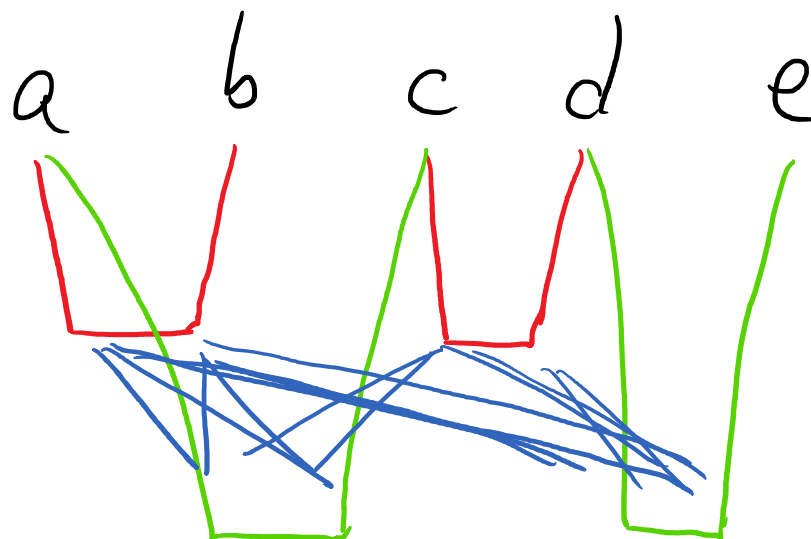
a ✗ b
 d ✗ c ✗

Observation: if G has *simwidth* 1 and ab, cd are independent edges of G , then any tree T for G satisfies $ab \mid cd$.

From Q to G : each quartet becomes a bunch of $2K_2$'s.

q_1 $ab \mid cd$

q_2 $ac \mid de$



Observation: if G has *simwidth* 1 and ab, cd are independent edges of G , then any tree T for G satisfies $ab \mid cd$.

From Q to G : each quartet becomes a bunch of $2K_2$'s.

$\Rightarrow$ If $Q \notin \text{UQC}$, then all trees fail to satisfy some $ab \mid cd$, and thus all trees have *simwidth* 2 or more.

Observation: if G has *simwidth* 1 and ab, cd are independent edges of G , then any tree T for G satisfies $ab \mid cd$.

From Q to G : each quartet becomes a bunch of $2K_2$'s.

$\Rightarrow$ If $Q \notin \text{UQC}$, then all trees fail to satisfy some $ab \mid cd$, and thus all trees have *simwidth* 2 or more.

$\Leftarrow$ If $Q \in \text{UQC}$, then by [Steel, 1993], there is a caterpillar T that satisfies all quartets of Q . We add every u_i^q next to p_i in T , resulting in a tree T' whose leaves are $V(G)$.

We see that [.....] and therefore T' has *simwidth* 1.

In fact, [.....] so T' has *linear- Omimwidth* 1, and so does G .

Observation: if G has *simwidth* 1 and ab, cd are independent edges of G , then any tree T for G satisfies $ab \mid cd$.

From Q to G : each quartet becomes a bunch of $2K_2$'s.

$\Rightarrow$ If $Q \notin \text{UQC}$, then all trees fail to satisfy some $ab \mid cd$, and thus all trees have *simwidth* 2 or more.

$\Leftarrow$ If $Q \in \text{UQC}$, then by [Steel, 1993], there is a caterpillar T that satisfies all quartets of Q . We add every u_i^q next to p_i in T , resulting in a tree T' whose leaves are $V(G)$.

We see that [.....] and therefore T' has *simwidth* 1.

In fact, [.....] so T' has *linear-omimwidth* 1, and so does G .

Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

For each $\sigma \in \{\text{simwidth}, \text{omimwidth}, \text{Omimwidth}\}$, it is NP-hard to decide whether a given graph G satisfies $\sigma(G) = 1$.

It is also NP-hard to decide whether *linear- σ* (G) = 1.

Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

It is NP-hard to decide whether a given graph G satisfies $mimwidth(G) \leq 2$.

It is also NP-hard to decide whether $linear\text{-}mimwidth(G) \leq 2$.

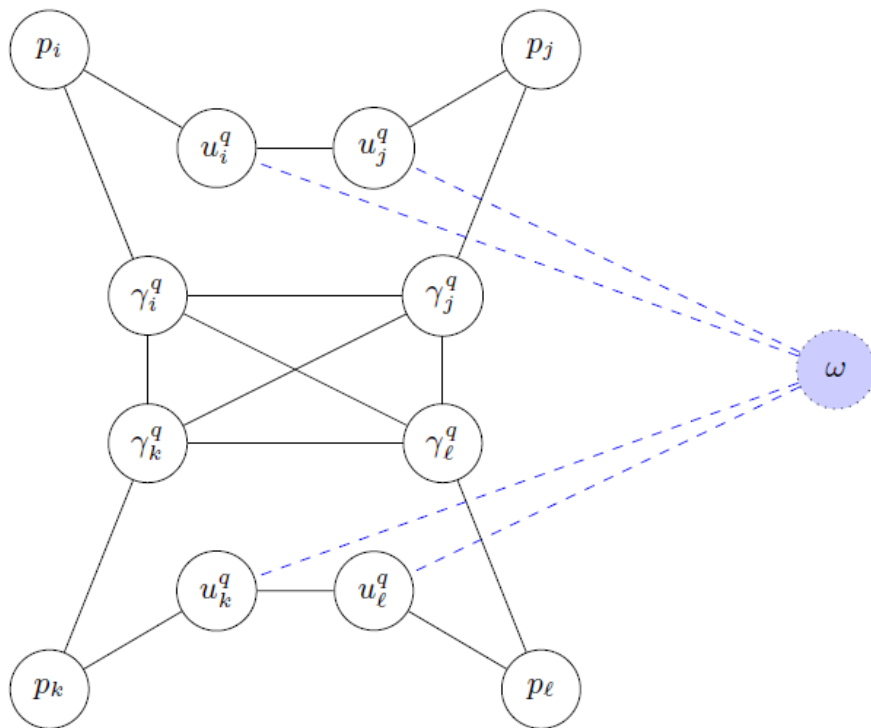
Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

It is NP-hard to decide whether a given graph G satisfies $mimwidth(G) \leq 2$.
It is also NP-hard to decide whether $linear\text{-}mimwidth(G) \leq 2$.

Another reduction from UQC. Previous one does not work.

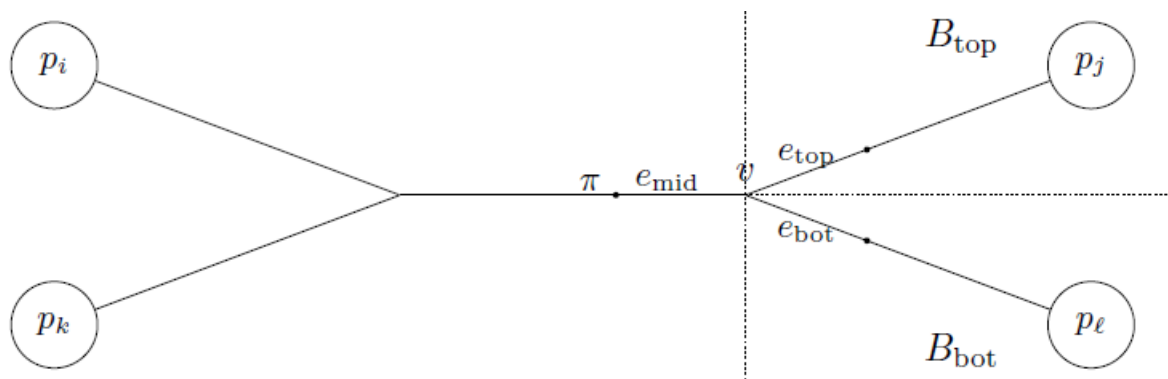
Intuitive reason: for $mimwidth = 1$, C_4 's encode imposed quartets.
The C_4 's in our construction can never be put together in a tree.

Similar, but more involved reduction.



Gadget for quartet
 $p_i p_j | p_k p_\ell$

Situation if wrong quartet, say
 $p_i p_k | p_j p_\ell$
 $mimwidth \geq 3$



Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

For each $\sigma \in \{\text{simwidth}, \text{omimwidth}, \text{Omimwidth}\}$, it is NP-hard to decide whether a given graph G satisfies $\sigma(G) = 1$.

It is also NP-hard to decide whether $\text{linear-}\sigma(G) = 1$.

Theorem [Dupré la Tour, L, Ndiaye, ICALP2026]

It is NP-hard to decide whether a given graph G satisfies $\text{mimwidth}(G) \leq 2$.

It is also NP-hard to decide whether $\text{linear-mimwidth}(G) \leq 2$.

All reductions have a linear number of vertices w.r.t. the UQC instance.

Old reductions show that UQC has no $2^{o(n)}$ time algorithm under ETH.

$\Rightarrow$ all above problems have **no $2^{o(n)}$ time algorithm under ETH.**

Open problems

1. Can UQC be useful for other problems?

Open problems

1. Can UQC be useful for other problems?
2. Is deciding $mimwidth(G) = 1$ NP-hard?

Open problems

1. Can UQC be useful for other problems?
2. Is deciding $mimwidth(G) = 1$ NP-hard?
3. Is deciding $linear-mimwidth(G) = 1$ NP-hard?

Open problems

1. Can UQC be useful for other problems?
2. Is deciding $mimwidth(G) = 1$ NP-hard?
3. Is deciding $linear-mimwidth(G) = 1$ NP-hard?
4. Maybe easier: is deciding $linear-mimwidth(G) = 1$ NP-hard on split graphs?
 - Got engulfed in that, made zero progress, would really want someone to answer this.

Open problems

1. Can UQC be useful for other problems?
2. Is deciding $mimwidth(G) = 1$ NP-hard?
3. Is deciding $linear-mimwidth(G) = 1$ NP-hard?
4. Maybe easier: is deciding $linear-mimwidth(G) = 1$ NP-hard on split graphs?
 - Got engulfed in that, made zero progress, would really want someone to answer this.
5. If deciding $\sigma\text{-width} \leq c$ is NP-hard, does that imply deciding $\sigma\text{-width} \leq c + 1$ is NP-hard?
 - Construction that transforms a graph of $mimwidth\ c$ to a graph of $mimwidth\ c + 1$